

# Lesson 3 Notes: Diagonal Matrix in C++ Using a Class

## 1. Main idea of Lesson 3

Lesson 3 continues the same diagonal matrix idea from Lesson 1 and Lesson 2.

The matrix type has not changed yet. We are still working with a **diagonal matrix**.

The main change is how we organize the code.

In Lesson 1, we used a simple 1D array.

In Lesson 2, we used a C `struct`.

In Lesson 3, we use a C++ `class`.

The diagonal matrix rule is still:

```
if (i == j)
```

That means:

- If `i == j`, the position is on the main diagonal.
- If `i != j`, the position is outside the main diagonal.
- Outside diagonal positions are treated as zero.
- We only store diagonal values.

The index formula is still:

```
index = i - 1
```

So the main goal of Lesson 3 is:

Take the diagonal matrix logic and write it in C++ using a class, constructor, destructor, and dynamic memory.

---

## 2. What you already know before Lesson 3

Before learning Lesson 3, you should already understand these ideas:

| Concept                | Meaning                                                                  |
|------------------------|--------------------------------------------------------------------------|
| Diagonal matrix        | A matrix where only the main diagonal values are stored.                 |
| Main diagonal          | Positions where row number and column number are the same.               |
| Condition              | <code>i == j</code>                                                      |
| Non-diagonal positions | Positions where <code>i != j</code>                                      |
| Structural zero        | A zero that exists because of the matrix rule, not because it is stored. |
| 1D storage array       | Stores only useful values instead of the full matrix.                    |
| Index formula          | <code>A[i - 1]</code>                                                    |
| Display                | Prints the full matrix using stored values and fake zeros.               |

Example diagonal matrix:

```
5 0 0 0
0 8 0 0
0 0 9 0
0 0 0 12
```

Stored form:

```
A = [5, 8, 9, 12]
```

The zeros are not stored.

They are created logically when needed.

### 3. Lesson 2 vs Lesson 3

In Lesson 2, the C version used a `struct` :

```
struct Matrix {
    int A[10];
    int n;
};
```

This grouped the diagonal array and matrix size together.

In Lesson 3, the C++ version uses a `class` :

```
class Diagonal {
private:
    int n;
    int *A;

public:
    Diagonal(int n);
    ~Diagonal();
    void set(int i, int j, int x);
    int get(int i, int j);
    void display();
};
```

The class stores:

```
int n;
int *A;
```

And it provides these operations:

```
set()
get()
display()
```

So the class keeps both the data and the operations together.

---

## 4. Why use a class?

A class helps us group related data and functions.

For a diagonal matrix, the related data are:

```
int n;    // matrix dimension
int *A;   // pointer to the 1D array
```

The related functions are:

```
set()  
get()  
display()
```

So instead of writing separate functions and passing the matrix again and again, the object handles its own data.

Example:

```
Diagonal d(4);  
  
d.set(1, 1, 5);  
d.set(2, 2, 8);  
d.display();
```

This means:

```
d is a diagonal matrix object.  
d stores values using its own set function.  
d prints itself using its own display function.
```

This is the basic object-oriented idea.

---

## 5. What is a class?

A class is a blueprint for creating objects.

In this lesson, the class is called:

```
Diagonal
```

The object may be created like this:

```
Diagonal d(4);
```

This means:

Create a diagonal matrix object named d.  
The matrix dimension is 4.

So `d` represents a  $4 \times 4$  diagonal matrix.

---

## 6. Class structure

The class from Lesson 3 is:

```
class Diagonal {  
private:  
    int n;  
    int *A;  
  
public:  
    Diagonal(int n);  
    ~Diagonal();  
    void set(int i, int j, int x);  
    int get(int i, int j);  
    void display();  
};
```

This class has two main sections:

1. `private`
  2. `public`
- 

## 7. The private section

```
private:  
    int n;  
    int *A;
```

The private section stores data that should not be directly accessed from outside the class.

Here:

```
int n;
```

stores the matrix dimension.

Example:

```
n = 4
```

means the matrix behaves like a  $4 \times 4$  matrix.

And:

```
int *A;
```

stores the address of the dynamic array that contains the diagonal values.

Example:

```
A = [5, 8, 9, 12]
```

Only diagonal values are stored in `A`.

---

## 8. Why private data is useful

If `n` and `A` are private, outside code cannot directly change them.

This is not allowed:

```
d.n = 5;  
d.A[0] = 100;
```

Why is that useful?

Because the class can protect the diagonal matrix rule.

Instead of directly changing the array, the user must call:

```
d.set(i, j, x);
```

Then `set()` can check:

```
if (i == j)
```

before storing the value.

So private data helps prevent invalid changes.

---

## 9. The public section

```
public:  
    Diagonal(int n);  
    ~Diagonal();  
    void set(int i, int j, int x);  
    int get(int i, int j);  
    void display();
```

The public section contains the functions that can be called from outside the class.

These are the allowed ways to use the object.

| Public member                | Purpose                                               |
|------------------------------|-------------------------------------------------------|
| <code>Diagonal(int n)</code> | Constructor. Creates the object and allocates memory. |
| <code>~Diagonal()</code>     | Destructor. Releases memory.                          |
| <code>set()</code>           | Stores a value if the position is diagonal.           |
| <code>get()</code>           | Returns a stored value or zero.                       |
| <code>display()</code>       | Prints the full visible matrix.                       |

---

## 10. Meaning of `int *A`

In Lesson 2, the array was fixed:

```
int A[10];
```

That means the array could hold only up to 10 diagonal values.

In Lesson 3, we use:

```
int *A;
```

This means `A` is a pointer.

It will point to a dynamic array created at runtime.

The actual array is created using:

```
A = new int[n];
```

This is better because the array size depends on the matrix size.

Example:

```
Diagonal d(4);
```

allocates 4 integer spaces.

```
Diagonal d(100);
```

allocates 100 integer spaces.

So the class can handle different matrix sizes.

---

## 11. Dynamic memory

Dynamic memory means memory is created while the program is running.

In C++, dynamic arrays are created using:

```
new int[n]
```

For example:

```
A = new int[5];
```

This creates an integer array with 5 positions.



For a diagonal matrix of size  $n$ , we only need  $n$  stored values.

So we allocate:

```
A = new int[n];
```

not:

```
A = new int[n * n];
```

Because a diagonal matrix does not store all  $n^2$  values.

---

## 12. Memory saving idea

A normal matrix stores every value.

For a  $5 \times 5$  matrix:

```
5 × 5 = 25 values
```

A diagonal matrix stores only the diagonal values:

```
5 values
```

So:

| Matrix type     | Stored values for $5 \times 5$ |
|-----------------|--------------------------------|
| Normal matrix   | 25                             |
| Diagonal matrix | 5                              |

In general:

| Matrix type     | Storage      |
|-----------------|--------------|
| Normal matrix   | $N^2$ values |
| Diagonal matrix | $N$ values   |

That is the memory advantage.

---

## 13. Constructor

A constructor is a special function that runs automatically when an object is created.

Constructor code:

```
Diagonal::Diagonal(int n) {  
    this->n = n;  
    A = new int[n];  
}
```

When we write:

```
Diagonal d(5);
```

this constructor runs automatically.

It does two jobs:

1. Stores the matrix size.
2. Allocates memory for the diagonal array.

---

## 14. Constructor step-by-step

Code:

```
Diagonal::Diagonal(int n) {  
    this->n = n;  
    A = new int[n];  
}
```

Step 1:

```
this->n = n;
```

This stores the parameter value inside the object.

Step 2:

```
A = new int[n];
```

This creates a dynamic array of size `n`.

Example:

```
Diagonal d(4);
```

After this:

```
n = 4  
A has 4 integer spaces
```

So the object is ready to store 4 diagonal values.

---

## 15. What does `this->n = n` mean?

This line is important:

```
this->n = n;
```

There are two different `n` names here.

The first one is:

```
this->n
```

This means the object's own `n` variable.

The second one is:

```
n
```

This means the parameter passed into the constructor.

So:

```
this->n = n;
```

means:

Store the constructor parameter `n` into the object's `n`.

Example:

```
Diagonal d(4);
```

The constructor receives:

```
n = 4
```

Then:

```
this->n = n;
```

stores 4 into the object's member variable.

---

## 16. Why do we need `this->`?

The class has a member variable named `n`:

```
int n;
```

The constructor also has a parameter named `n`:

```
Diagonal(int n)
```

Because both names are the same, we use `this->n` to clearly refer to the class member variable.

Without `this->`, the name may be confusing.

This is clearer:

```
this->n = n;
```

It means:

```
object's n = parameter n
```

---

## 17. Destructor

A destructor is a special function that runs automatically when the object is destroyed.

Destructor code:

```
Diagonal::~~Diagonal() {  
    delete[] A;  
}
```

The destructor releases the memory that was allocated by the constructor.

Because the constructor used:

```
new int[n]
```

we must release it using:

```
delete[] A;
```

---

## 18. Why the destructor is required

When we use dynamic memory, the memory does not automatically disappear in the same simple way as normal local variables.

The program asks for memory using:

```
new int[n]
```

So it should return that memory using:

```
delete[] A;
```

If we forget this, the program may cause a memory leak.

A memory leak means:

```
The program allocated memory but did not release it.
```

This is why the destructor matters.

Even though the class has only two data members, one of them points to heap memory.

So the destructor is necessary.

---

## 19. Important memory rule

The rule is:

```
new[] must be matched with delete[]
```

So:

```
A = new int[n];
```

must later be matched with:

```
delete[] A;
```

Wrong:

```
delete A;
```

Correct:

```
delete[] A;
```

Why?

Because `A` points to an array, not just one integer.

---

## 20. The `set()` function

The `set()` function stores a value in the matrix.

Code:

```
void Diagonal::set(int i, int j, int x) {  
    if (i == j) {  
        A[i - 1] = x;  
    }  
}
```

Meaning:

- `i` is the row number.
- `j` is the column number.
- `x` is the value to store.

The value is stored only if:

```
i == j
```

If `i != j`, the function does nothing.

---

## 21. Why `set()` checks `i == j`

In a diagonal matrix, only diagonal positions are stored.

Diagonal positions are:

```
M[1,1]  
M[2,2]  
M[3,3]  
M[4,4]
```

These positions all follow:

```
i == j
```

So `set()` must protect the matrix rule.

If someone tries:

```
d.set(1, 3, 99);
```

it should not store 99.

Why?

Because `(1,3)` is not on the diagonal.

That position must stay zero.

---

## 22. Dry run of `set()` for a diagonal position

Call:

```
d.set(3, 3, 9);
```

Step 1: Identify values.

```
i = 3  
j = 3  
x = 9
```

Step 2: Check condition.

```
i == j  
3 == 3  
true
```

Step 3: Calculate index.



```
index = i - 1  
index = 3 - 1  
index = 2
```

Step 4: Store value.

```
A[2] = 9;
```

So the value 9 is stored as the third diagonal value.

---

## 23. Dry run of `set()` for a non-diagonal position

Call:

```
d.set(3, 2, 7);
```

Step 1: Identify values.

```
i = 3  
j = 2  
x = 7
```

Step 2: Check condition.

```
i == j  
3 == 2  
false
```

Step 3: Do nothing.

The value 7 is ignored.

Why?

Because `(3,2)` is not on the main diagonal.

In a diagonal matrix, that position must be zero.

---

## 24. The `get()` function

The `get()` function returns the value at a matrix position.

Code:

```
int Diagonal::get(int i, int j) {  
    if (i == j) {  
        return A[i - 1];  
    }  
    return 0;  
}
```

Meaning:

- If the position is diagonal, return the stored value.
  - If the position is not diagonal, return zero.
- 

## 25. Why `get()` returns 0 for non-diagonal positions

In a diagonal matrix, non-diagonal positions are structural zeros.

Example:

```
M[1,4] = 0  
M[2,3] = 0  
M[4,1] = 0
```

These zeros are not stored in the array.

The program simply knows they are zero because of the matrix rule.

So when `i != j`, `get()` returns:

```
return 0;
```

without checking the array.

---

## 26. Dry run of `get()` for a diagonal position

Suppose:

```
A = [5, 8, 9, 12]
```

Call:

```
d.get(2, 2);
```

Step 1: Check condition.

```
2 == 2  
true
```

Step 2: Calculate index.

```
index = i - 1  
index = 2 - 1  
index = 1
```

Step 3: Return value.

```
return A[1];
```

Since:

```
A[1] = 8
```

Answer:

```
8
```

---

## 27. Dry run of `get()` for a non-diagonal position

Call:

```
d.get(1, 5);
```

Step 1: Check condition.

```
1 == 5  
false
```

Step 2: Return zero.

```
return 0;
```

The function does not check the array.

It already knows (1,5) is not on the diagonal.

---

## 28. The `display()` function

The `display()` function prints the full visible matrix.

Code:

```
void Diagonal::display() {  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= n; j++) {  
            if (i == j) {  
                cout << A[i - 1] << " ";  
            } else {  
                cout << "0 ";  
            }  
        }  
        cout << endl;  
    }  
}
```

This function uses two loops:

- Outer loop for rows.
- Inner loop for columns.

For each position `(i, j)`:

- If `i == j`, print the stored value.
  - If `i != j`, print `0`.
- 

## 29. Why `display()` uses nested loops

The stored array contains only diagonal values.

Example:

```
A = [5, 8, 9, 12]
```

But the displayed matrix must look like a full  $4 \times 4$  matrix:

```
5 0 0 0
0 8 0 0
0 0 9 0
0 0 0 12
```

To print this, the program must visit every row and every column.

That is why `display()` uses nested loops.

---

## 30. Dry run of `display()`

Assume:

```
n = 4
A = [5, 8, 9, 12]
```

Row 1:

```
(1,1): i == j, print A[0] = 5
(1,2): i != j, print 0
(1,3): i != j, print 0
(1,4): i != j, print 0
```

Output:

```
5 0 0 0
```

Row 2:

```
(2,1): i != j, print 0  
(2,2): i == j, print A[1] = 8  
(2,3): i != j, print 0  
(2,4): i != j, print 0
```

Output:

```
0 8 0 0
```

Row 3:

```
(3,1): i != j, print 0  
(3,2): i != j, print 0  
(3,3): i == j, print A[2] = 9  
(3,4): i != j, print 0
```

Output:

```
0 0 9 0
```

Row 4:

```
(4,1): i != j, print 0  
(4,2): i != j, print 0  
(4,3): i != j, print 0  
(4,4): i == j, print A[3] = 12
```

Output:

```
0 0 0 12
```

Full output:

```
5 0 0 0
0 8 0 0
0 0 9 0
0 0 0 12
```

---

## 31. Full Lesson 3 program

```
#include <iostream>
using namespace std;

class Diagonal {
private:
    int n;
    int *A;

public:
    Diagonal(int n) {
        this->n = n;
        A = new int[n];
    }

    ~Diagonal() {
        delete[] A;
    }

    void set(int i, int j, int x) {
        if (i == j) {
            A[i - 1] = x;
        }
    }

    int get(int i, int j) {
        if (i == j) {
            return A[i - 1];
        }
        return 0;
    }

    void display() {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                if (i == j) {
                    cout << A[i - 1] << " ";
                } else {
```

```

        cout << "0 ";
    }
}
cout << endl;
}
};

int main() {
    Diagonal d(4);

    d.set(1, 1, 5);
    d.set(2, 2, 8);
    d.set(3, 3, 9);
    d.set(4, 4, 12);

    cout << "Value at M[2][2]: " << d.get(2, 2) << endl;
    cout << "Value at M[1][4]: " << d.get(1, 4) << endl;

    cout << "\nFull matrix:" << endl;
    d.display();

    return 0;
}

```

## 32. Expected output

```

Value at M[2][2]: 8
Value at M[1][4]: 0

Full matrix:
5 0 0 0
0 8 0 0
0 0 9 0
0 0 0 12

```

## 33. Explanation of the full program

### Header file

```

#include <iostream>

```



This is needed because the program uses `cout` and `endl`.

## Namespace

```
using namespace std;
```

This allows us to write:

```
cout  
endl
```

instead of:

```
std::cout  
std::endl
```

## Class declaration

```
class Diagonal {
```

This creates a class named `Diagonal`.

## Private data members

```
private:  
    int n;  
    int *A;
```

These store the matrix size and the dynamic array.

## Constructor

```
Diagonal(int n) {  
    this->n = n;  
    A = new int[n];  
}
```

This initializes the object and allocates memory.

## Destructor

```
~Diagonal() {  
    delete[] A;  
}
```

This releases the memory when the object is destroyed.

## Set method

```
void set(int i, int j, int x)
```

This stores a value only if the position is diagonal.

## Get method

```
int get(int i, int j)
```

This returns a stored value for diagonal positions and zero for non-diagonal positions.

## Display method

```
void display()
```

This prints the full matrix.

## Main function

```
int main() {  
    Diagonal d(4);
```

This creates a  $4 \times 4$  diagonal matrix object.

Then:

```
d.set(1, 1, 5);  
d.set(2, 2, 8);  
d.set(3, 3, 9);  
d.set(4, 4, 12);
```

stores diagonal values.

Then:

```
d.get(2, 2)
```

returns 8.

And:

```
d.get(1, 4)
```

returns 0.

Finally:

```
d.display();
```

prints the full matrix.

---

## 34. Object creation dry run

Code:

```
Diagonal d(5);
```

What happens?

Step 1:

The constructor is called.

Step 2:

The parameter value is 5.

Step 3:

```
this->n = n;
```

stores 5 into the object's `n`.

Step 4:

```
A = new int[n];
```

creates an array of 5 integers.

Final object state:

```
n = 5  
A has 5 spaces
```

So the object can now represent a  $5 \times 5$  diagonal matrix.

---

## 35. Stored array vs visible matrix

Suppose the stored array is:

```
A = [3, 7, 4, 9, 6]
```

This represents the full matrix:

```
3 0 0 0 0  
0 7 0 0 0  
0 0 4 0 0  
0 0 0 9 0  
0 0 0 0 6
```

The computer stores only:

```
3, 7, 4, 9, 6
```

The zeros are not stored.

They are displayed or returned when needed.

---

## 36. Mapping matrix positions to array positions

For a  $5 \times 5$  diagonal matrix:

| Matrix position | Array position |
|-----------------|----------------|
| M[1,1]          | A[0]           |
| M[2,2]          | A[1]           |
| M[3,3]          | A[2]           |
| M[4,4]          | A[3]           |
| M[5,5]          | A[4]           |

Formula:

A[i - 1]

This formula is only used when:

i == j

---

## 37. Why the formula is `i - 1`

Matrix positions are written using 1-based indexing.

Example:

M[1,1]  
M[2,2]  
M[3,3]

But C++ arrays use 0-based indexing.

Example:

```
A[0]
A[1]
A[2]
```

So we subtract 1.

Examples:

```
M[1,1] -> A[1 - 1] -> A[0]
M[2,2] -> A[2 - 1] -> A[1]
M[3,3] -> A[3 - 1] -> A[2]
```

That is why we use:

```
A[i - 1]
```

---

## 38. Class member functions

A function inside a class is called a member function.

In this lesson, these are member functions:

```
void set(int i, int j, int x);
int get(int i, int j);
void display();
```

They belong to the `Diagonal` class.

So they are called using an object:

```
d.set(1, 1, 5);
d.get(2, 2);
d.display();
```

The object `d` calls its own functions.

---

## 39. Why methods can use `A` and `n` directly

Inside the class, member functions can directly use private data.

Example:

```
A[i - 1] = x;
```

and:

```
for (int i = 1; i <= n; i++)
```

This works because `set()`, `get()`, and `display()` are inside the same class as `A` and `n`.

Outside the class, direct access is not allowed.

Inside the class, direct access is allowed.

---

## 40. Lesson 2 vs Lesson 3 comparison

| Idea              | Lesson 2: C struct            | Lesson 3: C++ class                 |
|-------------------|-------------------------------|-------------------------------------|
| Data container    | <code>struct Matrix</code>    | <code>class Diagonal</code>         |
| Array type        | <code>int A[10]</code>        | <code>int *A</code>                 |
| Matrix size       | Stored in <code>n</code>      | Stored in private <code>n</code>    |
| Memory size       | Fixed maximum size            | Dynamic size                        |
| Memory allocation | Fixed array                   | <code>new int[n]</code>             |
| Memory release    | Not needed for fixed array    | <code>delete[] A</code>             |
| Functions         | Separate functions            | Member functions                    |
| Data protection   | Weak                          | Stronger using <code>private</code> |
| Set function      | Uses pointer to modify struct | Uses object member directly         |
| Get function      | Reads from struct             | Reads from object data              |

Short version:

Lesson 2 groups the data.  
Lesson 3 protects the data and manages memory.

## 41. Complexity of Lesson 3 operations

| Operation              | Time Complexity | Space Complexity | Reason                                         |
|------------------------|-----------------|------------------|------------------------------------------------|
| Object creation        | $O(1)$          | $O(N)$           | Allocates one array of size $N$ .              |
| <code>set()</code>     | $O(1)$          | —                | Uses one condition and one index calculation.  |
| <code>get()</code>     | $O(1)$          | —                | Uses one condition and maybe one array lookup. |
| <code>display()</code> | $O(N^2)$        | —                | Prints every visible matrix position.          |

## 42. Why object creation uses $O(N)$ space

The constructor allocates:

```
A = new int[n];
```

So the object stores  $n$  diagonal values.

That means storage is:

$O(N)$

It is not:

$O(N^2)$

because we do not store the full matrix.



### 43. Why `set()` is $O(1)$

The `set()` function does not use loops.

It only does:

1. One condition check.
2. One index calculation if needed.
3. One assignment if needed.

Code:

```
if (i == j) {  
    A[i - 1] = x;  
}
```

So `set()` takes constant time:

$O(1)$

---

### 44. Why `get()` is $O(1)$

The `get()` function does not use loops.

It only does:

1. One condition check.
2. One possible array access.

Code:

```
if (i == j) {  
    return A[i - 1];  
}  
return 0;
```

So `get()` takes constant time:

$O(1)$

---

## 45. Why `display()` is $O(N^2)$

The `display()` function prints every visible position.

For an  $N \times N$  matrix, there are:

$N^2$  positions

Even though only  $N$  values are stored, the display must print the full grid.

That is why display takes:

$O(N^2)$

---

## 46. Common beginner mistakes

### Mistake 1: Forgetting the destructor

Wrong:

```
~Diagonal() {  
}
```

Correct:

```
~Diagonal() {  
    delete[] A;  
}
```

Reason:

The constructor used `new int[n]`, so the destructor must release the memory.

---

### Mistake 2: Using `delete` instead of `delete[]`

Wrong:

```
delete A;
```

Correct:

```
delete[] A;
```

Reason:

`A` points to an array.

Arrays created with `new[]` must be released with `delete[]`.

---

### Mistake 3: Using `A[i]` instead of `A[i - 1]`

Wrong:

```
A[i] = x;
```

Correct:

```
A[i - 1] = x;
```

Reason:

Matrix positions start at 1, but array indexes start at 0.

---

### Mistake 4: Storing non-diagonal values

Wrong idea:

```
set(1, 3, 99) should store 99 somewhere.
```

Correct idea:

```
set(1, 3, 99) should do nothing.
```

Reason:

`1 != 3`, so the position is not diagonal.

---

### Mistake 5: Thinking display means all values are stored

The display may show:

```
5 0 0 0
0 8 0 0
0 0 9 0
0 0 0 12
```

But the actual stored array is only:

```
A = [5, 8, 9, 12]
```

The zeros are printed, not stored.

---

### Mistake 6: Forgetting that `private` protects data

Wrong idea:

```
I should directly change d.A[0].
```

Correct idea:

```
Use d.set(1, 1, value).
```

Reason:

The class should control how its data changes.

---

## 47. Beginner mental model

Think of the class as a small machine.

Inside the machine:

```
n = matrix size  
A = stored diagonal values
```

Outside the machine, the user presses buttons:

```
set()  
get()  
display()
```

The user does not directly touch the internal array.

This protects the matrix rule.

So the object works like this:

```
User asks to store a value.  
The class checks if the position is diagonal.  
If yes, it stores the value.  
If no, it ignores the value.
```

And:

```
User asks to get a value.  
The class checks if the position is diagonal.  
If yes, it returns the stored value.  
If no, it returns 0.
```

---

## 48. Practice example 1

Suppose:

```
Diagonal d(4);
```

Then we call:

```
d.set(1, 1, 10);  
d.set(2, 2, 20);
```

```
d.set(3, 3, 30);  
d.set(4, 4, 40);
```

Stored array:

```
A = [10, 20, 30, 40]
```

Full visible matrix:

```
10 0 0 0  
0 20 0 0  
0 0 30 0  
0 0 0 40
```

Questions:

```
d.get(1, 1)  
d.get(3, 3)  
d.get(2, 4)  
d.get(4, 1)
```

Answers:

| Operation                | Answer | Reason                                        |
|--------------------------|--------|-----------------------------------------------|
| <code>d.get(1, 1)</code> | 10     | Diagonal position, return <code>A[0]</code> . |
| <code>d.get(3, 3)</code> | 30     | Diagonal position, return <code>A[2]</code> . |
| <code>d.get(2, 4)</code> | 0      | Non-diagonal position.                        |
| <code>d.get(4, 1)</code> | 0      | Non-diagonal position.                        |

## 49. Practice example 2

Suppose:

```
Diagonal d(5);
```

Stored array:

```
A = [2, 4, 6, 8, 10]
```

This represents:

```
2 0 0 0 0
0 4 0 0 0
0 0 6 0 0
0 0 0 8 0
0 0 0 0 10
```

Questions and answers:

| Operation                    | Answer                         | Reason                     |
|------------------------------|--------------------------------|----------------------------|
| <code>d.get(1, 1)</code>     | 2                              | Return <code>A[0]</code> . |
| <code>d.get(5, 5)</code>     | 10                             | Return <code>A[4]</code> . |
| <code>d.get(3, 4)</code>     | 0                              | Non-diagonal.              |
| <code>d.get(5, 2)</code>     | 0                              | Non-diagonal.              |
| <code>d.set(3, 3, 99)</code> | Stores 99 at <code>A[2]</code> | Diagonal.                  |
| <code>d.set(2, 5, 77)</code> | Ignored                        | Non-diagonal.              |

## 50. Key rules from Lesson 3

### Class design

```
class Diagonal {
private:
    int n;
    int *A;
public:
    Diagonal(int n);
    ~Diagonal();
    void set(int i, int j, int x);
    int get(int i, int j);
    void display();
};
```

### Diagonal condition

```
i == j
```

### Index formula

```
A[i - 1]
```

### Constructor responsibility

```
Store n and allocate memory.
```

### Destructor responsibility

```
Release memory.
```

### Set rule

```
If i == j, store x in A[i - 1].  
If i != j, do nothing.
```

### Get rule

```
If i == j, return A[i - 1].  
If i != j, return 0.
```

### Display rule

```
Loop through every visible matrix position.  
Print stored values on the diagonal.  
Print zero everywhere else.
```

---

## 51. Lesson 3 final summary

Lesson 3 teaches how to implement a diagonal matrix using a C++ class.

The matrix rule is the same as before:



```
i == j
```

Only diagonal positions are stored.

The array formula is still:

```
A[i - 1]
```

The new ideas are:

```
class
private data
public methods
constructor
destructor
dynamic memory
new int[n]
delete[] A
member functions
```

The constructor creates the internal array.

The destructor releases the internal array.

The `set()` method stores only diagonal values.

The `get()` method returns stored diagonal values or zero.

The `display()` method prints the full matrix using real values and fake zeros.

The most important memory rule is:

```
If you use new[], you must use delete[].
```

The most important matrix rule is:

```
Only positions where i == j are stored.
```

## 52. Exit questions

You should be able to answer these before moving to Lesson 4:

1. What is the purpose of the `Diagonal` class?
  2. Why are `n` and `A` private?
  3. What does `int *A` mean?
  4. Why do we use `new int[n]`?
  5. What does the constructor do?
  6. What does `this->n = n` mean?
  7. What does the destructor do?
  8. Why do we use `delete[] A`?
  9. What does `set(3, 3, 9)` do?
  10. What does `set(3, 2, 7)` do?
  11. What does `get(2, 2)` return?
  12. Why does `get(1, 5)` return zero?
  13. Why does `display()` take  $O(N^2)$  time?
  14. What is the index formula for diagonal values?
  15. Why is a diagonal matrix stored in  $O(N)$  space?
- 

## 53. Short answers to exit questions

1. The `Diagonal` class stores and manages a diagonal matrix.
2. They are private to protect the matrix data.
3. `int *A` means `A` is a pointer to a dynamic integer array.
4. We use `new int[n]` to create an array of the correct size at runtime.
5. The constructor stores `n` and allocates memory.
6. It means the object's `n` receives the constructor parameter `n`.
7. The destructor releases the allocated memory.
8. We use `delete[] A` because `A` was created using `new[]`.
9. It stores 9 at `A[2]`.
10. It does nothing because `(3, 2)` is not diagonal.
11. It returns the value stored at `A[1]`.
12. Because `(1, 5)` is a structural zero.
13. Because it prints every position in the full  $N \times N$  matrix.
14. The formula is `A[i - 1]`.
15. Because only `N` diagonal values are stored, not  $N^2$  values.